MPP001 Computer Labs

Simon Martin

2027-11-09

Table of contents

1 Preface **4**

2 Introduction **5**
2.1 Python on you own computer . . . 5
2.1.1 Python for research . . . 5

3 Welcome to your Python journey **6**
3.1 Notebooks vs code files . . . 6
3.1.1 Notebook quickstart . . . 6
3.2 Text blocks in notebooks . . . 8
3.3 Next steps . . . 9
3.4 Simple maths in python . . . 9
3.4.1 Beyond simple maths . . . 9
3.4.2 `as library_name` . . . 10

4 Introduction **11**
4.1 Starting a python session on Lab PCs. . . . 11
4.2 Python on you own computer . . . 11
4.2.1 Python for research . . . 12

5 First steps in Python **13**
5.1 Notebook-based python . . . 13
5.1.1 Exercise: . . . 13
5.1.2 Quick way to run cells . . . 13
5.1.3 File extensions . . . 13
5.2 Simple maths in python . . . 14
5.2.1 Beyond simple maths . . . 14
5.2.2 `as library_name` . . . 15
5.2.3 Exercises . . . 15
5.3 Read the docs . . . 16

6 Lab learning plan **17**
6.1 Gen AI use . . . 17

7 Lab 1: Engineering Calculations with Python **18**
7.1 Learning Objectives . . . 18

7.2 Running python code – notebooks . . . 18
7.3 Starting Jupyter . . . 18
7.4 Storing values in variables . . . 23
7.5 Functions . . . 24
7.6 Formatting results . . . 24
7.7 Assessment . . . 25

8 Programming (in python) 26
8.1 Algorithms . . . 26
8.1.1 Example: how many cans of paint? . . . 27

9 Lab 2 The building blocks of computer code 29
9.1 Conditional statements . . . 29
9.2 Loops . . . 29
9.2.1 for loops . . . 30
9.2.2 while loops . . . 30
9.2.3 `break` and `continue` . . . 31
9.3 Functions . . . 32
9.4 Finding out more about python . . . 33

10 Week 2 Lab 34
10.1 Actions . . . 34

11 Lab 4: Engineering Data Analysis with Pandas 35
11.1 Pandas . . . 35
11.2 Getting started . . . 35
11.2.1 A bit of computing terminology: methods . . . 36
11.2.2 back to the code. . . . 37
11.3 Filtering data . . . 38
11.4 Plotting with Pandas . . . 39
11.5 Accessing dataframe elements . . . 41
11.5.1 loc[] . . . 42
11.5.2 iloc[] . . . 43
11.6 Working with numpy . . . 44
11.7 Creating copies . . . 44

References 46

1 Preface

These notes are created using the Quarto system. In order to aid accessibility the main font in the PDF version is Atkinson Hyperlegible and the text blocks are left-justified where possible.

To learn more about Quarto books visit https://quarto.org/docs/books.

2 Introduction

These notes are written from th epoint of view of working with a PC in one of LU's computer labs. We'll be using the Jupyter Lab environment (app). You can install essentially the same software on your own computer for free – see below.

2.1 Python on you own computer

You can install Python on your own computer. The computers in the teaching labs use the Anaconda distribution https://www.anaconda.com. It contains a lot of useful python packages and a lot more. It is free for educations use and installers are available for Windows, MacOS, and Linux. If disk space is a premium for you then search on the Anaconda site for the 'miniconda' version which contains the basics and there is an easy way to add any extras that you need.

2.1.1 Python for research

The Anaconda Python distribution is **NOT** free for research (or commercial) use. I use the conda-forge distribution instead, this provides a very similar experience to the Anaconda distribution.

Other versions of python are available. If you do not use Anaconda/conda-forge then do make sure you are installing a one of the Python 3 variations (I advise using version 3.8 or higher). Python 2 is still available, but has some significant differences.

MacOS (and many Linux versions) come with a version of Python however these versions often lack the science libraries needed for this module and it is generally easier to install Anaconda/conda-forge.

Anaconda also comes with a range of tools for editing and running python code. Spyder is probably the best (free) choice for editing science/engineering code. You can install it separately or as part of the Anaconda installation. Another excellent choice is pyCharm.

3 Welcome to your Python journey

This is a short introduction to python and using it on University computers (and your own computer should you wish to).

This documentation is available in website form or as a single PDF. If you are viewing the website version then the links on the left will help you navigate between chapters and the links on the right allow you to navigate within a webpage.

3.1 Notebooks vs code files

Traditionally, python coding has been done with individual files (usually with a `.py` extension). These files are edited then sent to a python interpreter to execute the code. More recently, a system based on notebooks has become very popular, particularly for science/engineering applications.

In this module we will be using `.ipynb` format notebooks. I prefer the notebook approach for small programming tasks, but still use the `.py` approach for more involved work. The lab material and coursework submissions will be jupyter notebooks, but there will be examples of `.py` files for you to explore across the module.

3.1.1 Notebook quickstart

If you have not used Jupyter notebooks before then read this before going any further: Jupyter notebook quickstart

The code below is an example of python code in a Jupyter notebook - the leading notebook system for python. You can download a copy of this notebook from the Python section of the MPP001 Learn page – it is called 'BasicNotebook.ipynb'

The code has been split into three sections–these are referred to as cells.

- The first cell imports (brings in) external code libraries to extend what python can do.
- The second cell sets up some parameters
- The third cell does some working out and displays the results

This general format is quite common for a notebook. There are often more cells, but the general layout remains the same.

Note: in python the # symbol is used to mark what follows on the line as being a comment. This means that anything to the right of the symbol is ignored by python. This allows you to put explanatory notes in your code. It is also a quick way to turn off one or more lines of code when you are testing/debugging it.

```
# The code uses the stats library to generate an array of 1000 points
# distribution with a mean of 20 and a standard deviation of 4

# Import required libraries
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt
```

```
# Specify the properties of the normal distribution
n_sample = 1000
mean = 20
sigma = 4
```

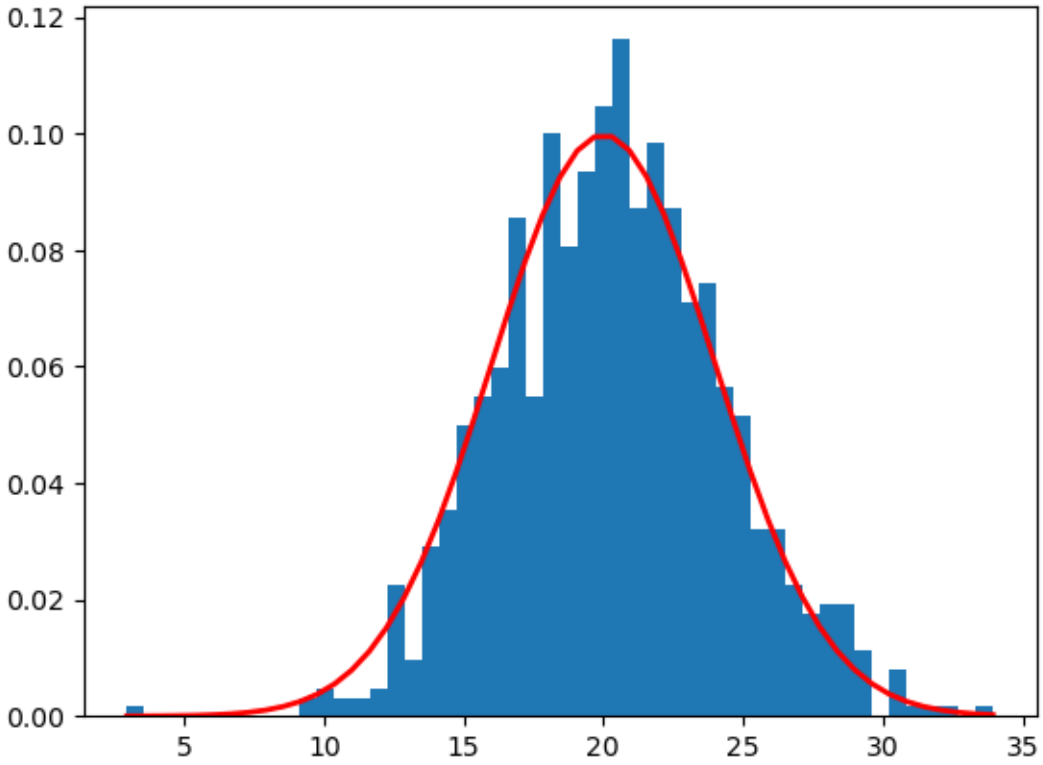
```
# Generate an array of data belonging to the normal distribution
rand_dist1 = stats.norm.rvs(loc = mean, scale = sigma, size = n_sampl

# Check that the data meets expectations
print(f"Mean = {np.mean(rand_dist1):,.4f} and stddev = {np.std(rand_d
print(stats.normaltest(rand_dist1))

# Plot the histogram of the data and the distribution function
count, bins, ignored = plt.hist(rand_dist1, 50, density=True)

plt.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) * np.exp( - (bins - mea
plt.show()
```

```
Mean = 20.3478 and stddev = 3.9695
NormaltestResult(statistic=2.74838621513032, pvalue=0.253043692492792
```



3.2 Text blocks in notebooks

You can also have blocks of text in notebooks. These allow various formatting options:

- **Bold**
- *Itallic*
- `highlighted`

And maths is quite easy for example `$ E = mc^2$` generates: $ E = mc^2$ - this is 'in-line' maths. Pairs of dollar symbols will generate centred maths on its own line e.g. `$$ E = mc^2$$` gives

$$E = mc^2$$

For text blocks Jupyter Notebooks use a formatting system called Markdown – it can do a lot more than shown above. Have a look here for more details:
https://www.markdownguide.org/basic-syntax/ (or here Jupyter markdown docs)

3.3 Next steps

It is now time to start getting going with Python coding.

Notebooks with worked examples to explore and blank sections for you to complete will be provided via the Module's Team.

3.4 Simple maths in python

Python has a range of standard mathematical functions as sumarised in the table below. If you have used something like Excel for maths then you should be familiar with them. Try to predict the outcomes of the examples given before entering them into your python system. Do they give the results you expected?

Symbol	operation	example
+	addition	3+2
-	substraction	2-3
*	multiplication	3*2
/	division	2/3
**	exponent	3**2

There are a range of other maths operators as well, but this is enough for now.

3.4.1 Beyond simple maths

Given the similarity to Excel for the operators above, what do you think the following will produce?

```
cos(0.0)
```

Start a new notebook and try the code in a block.

You probably got something like:

```
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    cos(0)
NameError: name 'cos' is not defined
```

Which is a longwinded way of the python system telling you it has no function `cos()`. One of the key features of python is that it is easily extendable. T:

```
import numpy as np
np.cos(0.0)
```

The first line in the block above tells python to use the module called numpy (often pronounced num–pie) and all references to its contents will be preceded by `np.` for example `np.sin(3.141)`. Numpy is an extensive maths library that is very well optimised and we will use it a lot in MPP001–along with a lot of other libraries.

3.4.2 as library_name

The `as np` can be neglected and then to access numpy's functions `numpy.` will need to be added instead of 'np'. However it is very common to use `as library_name` because the full library name can be quite long and this can make the card hard to read (and longer to type). Formally the `as` sets up an alias to the original name of the library so you.

`np` is used by convention for the numpy library, you can use any text identifier you like, but remember that you and others need to be able to follow the code so keep it simple. For instance you could use `include numpy as Lb0r0R0cks` at the start of your code and then, say 100 lines later, have `Lb0r0R0cks.cos(XVal)` which would be very difficult to follow and you would have to search for what it means.

In this module you'll meet a few other widely used libraries that, like numpy, have pretty much universally accepted, standard names:

Library	Purpose	Standard short name
Scipy	A collection of scientific computing methods	`sp`
Pandas	Data manipulation and analysis	`pd`
Scikit Learn	A collection of machine learning methods	`sklearn`

There are plenty of other examples - look out for them in your reading/learning for this module.

4 Introduction

This part of the guide provides a quick overview of how to get Python running on one of the University PCs and a very short introduction to running python code.

4.1 Starting a python session on Lab PCs.

Once you are logged in to a lab PC you can start a python session by using the Start Menu – Click on the Anaconda Folder and select the Jupyter Lab app. The Jupyter Notebook app is similar but has less functionality.

There is also a Python section in the Start Menu. This gives access to other versions, but it is much better to use the method above as it works reliably and you will get the same apps etc. on your own computer if you install Anaconda (see below).

> Make sure you have the right Python version
>
> You might be inclined to use the search box to find/start python items. The problem is that this shows up items relating to another version of python (used by SPSS) that is not set up for general use. Worse, it uses an out of date version of python that is not compatible with much of the code used in this module.

4.2 Python on you own computer

You can install Python on your own computer. The computers in the teaching labs use the Anaconda distribution https://www.anaconda.com. It contains a lot of useful python packages and a lot more. It is free for educations use and installers are available for Windows, MacOS, and Linux. If disk space is a premium for you then search on the Anaconda site for the 'miniconda' version which contains the basics and there is an easy way to add any extras that you need.

4.2.1 Python for research

The Anaconda Python distribution has a mixed history for non-educational use. For research I use the conga-forge distribution instead, this provides a very similar experience to the Anaconda distribution.

Other versions of python are available. If you do not use Anaconda/conda-forge then do make sure you are installing a one of the Python 3 variations (I advise using version 3.10 or higher). Python 2 is still available, but has some significant differences and you may run into problems using it these days.

MacOS (and many Linux versions) come with a version of Python however these versions often lack the science libraries needed for this module and it is generally easier to install Anaconda/conda-forge.

Anaconda also comes with a range of tools for editing and running python code. Spyder is probably the best (free) choice for editing science/engineering code. You can install it separately or as part of the Anaconda installation. Another excellent choice is pyCharm.

5 First steps in Python

The easiest way to get started with python is to launch the Anaconda Navigator app. You then have to choose whether to use the file based ('.py') or notebook approach. I recommend you take the time to try both.

5.1 Notebook-based python

If you didn't fancy the guide given in the link on the Welcome page then have a quick look here: Jupyter notebooks

5.1.1 Exercise:

Modify the example in the link above so that it gives the "Hello World!" greeting.

5.1.2 Quick way to run cells

Clicking the Run cell button is fine, but you can save time by pressing the Shift key at the same time as pressing the Return key. This will run whichever cell the cursor is in.

This means that you can go back to a cell, edit it, and re-run just that cell. If you use a '.py' file approach then all the code has to be re-run when you make a modification.

5.1.3 File extensions

So far, we have typed in code to a python session and observed the results. For anything involving more than two or three lines of code we need a way to save/recall code. Jupyter files have the extension `.ipynb`

5.2 Simple maths in python

Python has a range of standard mathematical functions as sumarised in the table below. If you have used something like Excel for maths then you should be familiar with them. Try to predict the outcomes of the examples given before entering them into your python system. Do they give the results you expected?

Symbol	operation	example
+	addition	3+2
-	substraction	2-3
*	multiplication	3*2
/	division	2/3
**	exponent	3**2

There are a range of other maths operators as well, but this is enough for now.

5.2.1 Beyond simple maths

Given the similarity to Excel for the operators above, what do you think the following will produce?

```
cos(0.0)
```

Try it in the Spyder console.

You probably got something like:

```
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    cos(0)
NameError: name 'cos' is not defined
```

Which is a longwinded way of the python system telling you it has no function `cos()`. One of the key features of python is that it is easily extendable. Try the following code snippet:

```
import numpy as np
np.cos(0.0)
```

The first line in the block above tells python to use the module called numpy (often pronounced num–pie) and all references to its contents will be preceded by `np.` for example `np.sin(3.141)`. Numpy is an extensive maths library that is very well optimised and we will use it a lot in MPP001–along with a lot of other libraries.

5.2.2 as library_name

The `as np` can be neglected and then to access numpy's functions `numpy.` will need to be added instead of 'np'. However it is very common to use `as library_name` because the full library name can be quite long and this can make the card hard to read (and longer to type). Formally the `as` sets up an alias to the original name of the library so you.

`np` is used by convention for the numpy library, you can use any text identifier you like, but remember that you and others need to be able to follow the code so keep it simple. For instance you could use `include numpy as Lb0r0R0cks` at the start of your code and then, say 100 lines later, have `Lb0r0R0cks.cos(XVal)` which would be very difficult to follow and you would have to search for what it means.

In this module you'll meet a few other widely used libraries that, like numpy, have pretty much universally accepted, standard names:

Library	Purpose	Standard short name
Scipy	A collection of scientific computing methods	`sp`
Pandas	Data manipulation and analysis	`pd`
Scikit Learn	A collection of machine learning methods	`sklearn`

There are plenty of other examples - look out for them in your reading/learning for this module.

5.2.3 Exercises

1. Investigate the operator precedence in python.
2. Experiment with the maths functions in numpy-are all the ones you expect there? Do they work the way you expect?
3. Modern python has a maths library called "math" that provides similar maths functions to numpy. Numpy provides a lot of extra functionality, but for most things there is not much to choose between them. Repeat your numpy maths experiments with the math module. Do you see any different answers?

5.3 Read the docs

Whilst discussing the standard libraries, it is worth pointing out that they have excellent online documentation. For example, for Pandas look here: https://pandas.pydata.org/pandas-docs/stable/user_guide/index.html. Smaller libraries are likely to use the ReadTheDocs web system for their documentation e.g. https://pytube.io/en/latest/index.html. This system provides a standard look for documentation that makes it easier to find you way round new libraries.

6 Lab learning plan

Over the next few weeks we will cover thes topics listed below which are deisgned to help you learn programming in python with an emphasis on reproducable workflows that are required in engineering and related careers:

0. Basics of python and notebooks
1. Write, execute and debug Python programs.
2. Use variables, functions, conditionals and loops appropriately.
3. Work with arrays and tabular engineering data.
4. Use Python libraries including NumPy, Pandas and Matplotlib.
5. Implement simple engineering calculations and simulations.
6. Developing reproducible computational workflows suitable for MSc-level engineering analysis.

1 to 6 will have assessed exercises associated with them. Together these exercises are worth 10% of the module.

This assessment will be done by submitting each week's notebook to Gradescope via the appropriate submission point on Learn. Gradescope will evaluate your code, checking that it performs the required functions and check for similarity with the work of others and any other unusual patterns.

6.1 Gen AI use

You should avoid using AI (ChatGPT, Claude, etc.) with the lab problems as **this will significantly interfere** with your learning of programming skills. Some of the notebooks, particularly those towards the end of the course will use Gradescope to embed hidden tests that will prevent GenAI producing useful results. If you have skipped the learning process then you are likely to struggle with solving the notebook problems.

7 Lab 1: Engineering Calculations with Python

7.1 Learning Objectives

At the end of this exercise set you should be able to:

- Run Python code in notebooks
- Store values in variables
- Use mathematical operators
- Format numerical output
- Understand the relationship between engineering equations and code

7.2 Running python code – notebooks

In this module we will be using Jupyter Notebooks as the environment to run python code. Notebooks are useful ways to develop code alongside embeded notes, equations, images. They are excellent ways to produce reports, presentations, and even entire books.

7.3 Starting Jupyter

On a University PC: In the search box type `Anaconda`. Amongst the options you should see 'Anaconda Navigator' - select this.

Once Anaconda has started: Click on 'Launch' in the Jupyter Lab box. This will start a browser window that looks something like this:

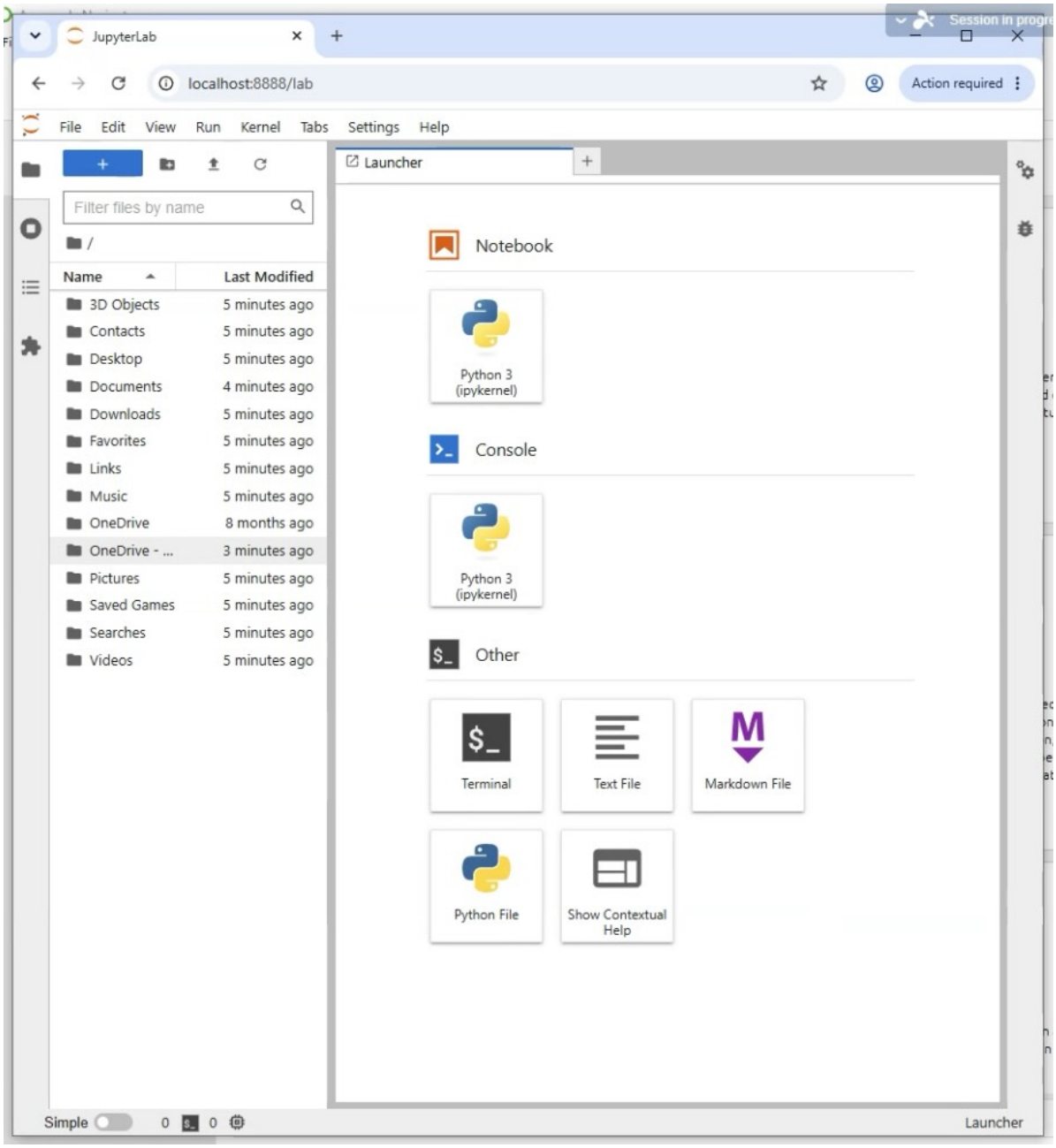
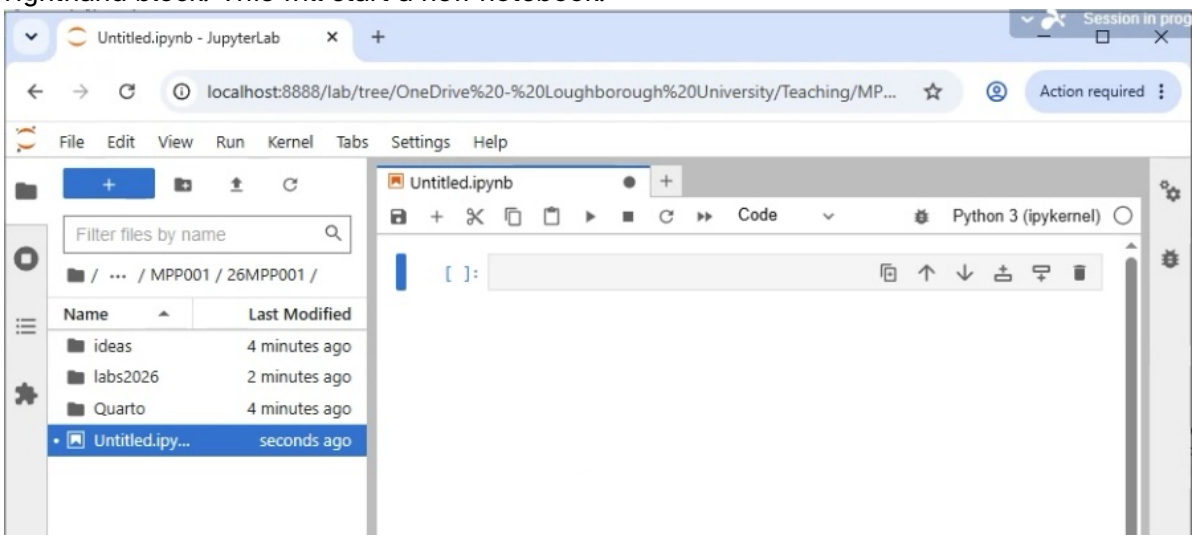


Figure 7.1: Screenshot of Jupyter start screen

The right hand block on this screen allows you to create and start editing files of different types. The left hand block determines where those files will end up.

> **Important**
>
> on startup Jupyter defaults to the main user folder on the PC - you must not save your files here - they are wiped at 6am **every day**. Instead click on `OneDrive-Loughborough University` to take you to your OneDrive space. I advise creating a folder there for your MPP001 work.

Once in your MPP001 folder you can click on the 'Python 3' square just below Notebook in the righthand block. This will start a new notebook:



Click in the light-grey box to begin to write code. An easy starter is:

```
print('Hello world!')`
```

Press Shift and Return at the same time to execute the code in the block. You should get something like this:

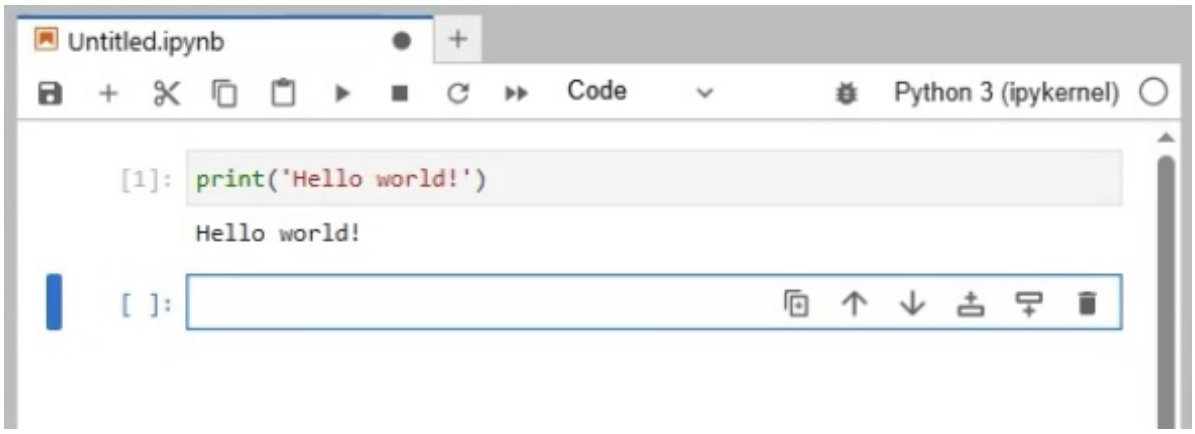


Figure 7.2: Screenshot of notebook with 'Hello world!' code

Notice that the box you typed in now has a number - this tells you that the code in it has been executed and when there is more than one code block it lets you work out the order of running the blocks. Second the text 'Hello world!' has appeared below the block – this is the output of the python code. Finally, a new empty block awaits your input.

If you had just pressed return then you would have gone to a new line in the block and you could have added more code.

You can go back to a cell to modify it and/or re-run it. To do this, just click in the cell.

Blocks in notebooks can be one of 3 kinds: Code - the default, Markdown – useful for text, images, etc., and Raw - we are unlikely to use this. You can change a block type by clicking on the word Code in the top-bar of the window.

You can find out more about notebooks at this link

The code below is an example of a small Jupyter notebook. The code has been split into three cells. It could have been put in a single cell, but dividing code up like this makes it easier to follow and much easier to maintain or adapt.

- The first cell imports (brings in) external code libraries to extend what python itself can do.
- The second cell sets up some parameters
- The third cell does some working out and displays the results

This general format is quite common for a notebook. There are often more cells, but the general layout remains the same.

Note: in python the # symbol is used to mark what follows on the line as being a comment. This means that anything to the right of the symbol is ignored by python. This allows you to put

explanatory notes in your code. It is also a quick way to turn off one or more lines of code when you are testing/debugging it.

```
# The code uses the stats library to generate an array of 1000 points
# distribution with a mean of 20 and a standard deviation of 4

# Import required libraries
from scipy import stats # a suite of statistics related functions
import numpy as np # numpy provides numerical functions and storage s
import matplotlib.pyplot as plt # plotting functions inspired by Matl
```

```
# Specify the properties of the normal distribution
n_sample = 1000
mean = 20
sigma = 4
```

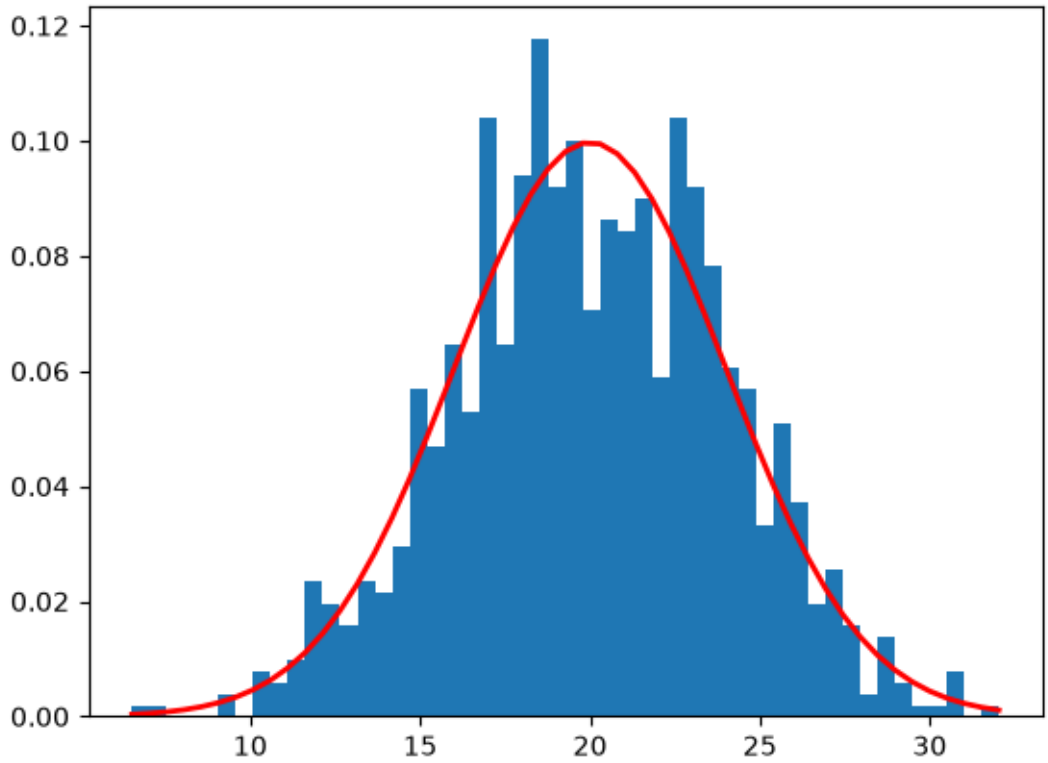
```
# Generate an array of data belonging to the normal distribution
rand_dist1 = stats.norm.rvs(loc = mean, scale = sigma, size = n_sampl

# Check that the data meets expectations
print(f"Mean = {np.mean(rand_dist1):.4f} and stddev = {np.std(rand_di
print(stats.normaltest(rand_dist1))

# Plot the histogram of the data and the distribution function
count, bins, ignored = plt.hist(rand_dist1, 50, density=True)

plt.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) * np.exp( - (bins - mea
plt.show()
```

```
Mean = 20.05 and stddev = 3.9996
NormaltestResult(statistic=np.float64(1.6929087397738105), pvalue=np.
```



7.4 Storing values in variables

In programming we almost always need to store values e.g. in the above code the properties of the normal distribution were specified in the second block. These values are then used in the third block by referring to the variable names. The general format for assigning variables is

```
variable_name = value
```

Very often the value will be a number (e.g. `lab_length = 2`) however other types of information can be stored e.g.

```
name = 'Simon'
print(f'My name is {name}')
```

> **Note**
>
> the pair of '...' around the value tell python that the value is a string (text) rather than the name of another variable. Python has a range of different variable types - you should be

able to spot them as you go along. Python can usually work out the type from the context of the code.

> **Note**
>
> The `f` at the beginning of the string is important – there is more about later in this chapter.

7.5 Functions

Very often in programming we need to do use the same block of code multiple times, perhaps with different start values. One of the ways of achieving this is a function. Python has a lot of built in functions e.g. `print()` in the code above we'll meet many more as the learning progresses. Most programming languages have a way of defining functions (blocks of code) for re-use so that what can be done can be extended.

In python we define a function as follows:

```
def function_name(parameters):
    # code block (the indenting of the code is important)

    #
    return value_returned # if any
```

`function_name` is the identifier used to reference, or 'call', the function. `parameters` is a comma separated list of one or more values that are used by the function's code block. The code block can produce output as it runs or return a value to the point in the code that it was called.

7.6 Formatting results

In the third cell above the line

```
# Check that the data meets expectations
print(f"Mean = {np.mean(rand_dist1):,.4f} and stddev = {np.std(rand_d
```

results in this output: `Mean = 19.8415 and stddev = 3.9406`

Notice that the numerical results are both given to 4 decimal places. This is no accident, the code in that `print()` statement is set up to control the format of the output. This process

starts with the `f` just to the right of the first bracket. This tells Python that the string (text) that follows is likely to contain placeholders `{}` which can contain variables, code, and formatting information.

The first placeholder above has two parts: `np.mean(rand_dist1)` this calls the numpy library function `mean()` with `rand_dist1` as its parameter value. It will return a number with a lot of decimal places. The part of the placeholder after the colon is a formatter `.4f` this tells Python to give the result to 4 decimal places. If you (quite sensibly) prefer to limit the number of significant figures then use `g` in place of the `f`. Try changing the formatters in the code above. You can find out more about formatters at this page. They can do more than control number precision.

The second placehold has a very similar structure. You can modify one and compare its new output to the other one to help get a feel for what the various formatters do.

7.7 Assessment

The work for the first week is designed to help you get used to: Assigning values, maths functions, creating your own functions.

Download the Week 1 notebook from the Learn page and attempt the programming problems given.

8 Programming (in python)

8.1 Algorithms

There are two absolutely key steps in writing any computer program (this applies to any programming language):

1. Understanding what it is you want to do
2. Breaking this down into individual steps

If you can do these for the problem in hand then the process of writing the code is usually straightforward.

Put together these steps allow you to develop an algorithm for the problem you are working on. In formal computer science there is more to it than this, but we need to be pragmatic and get on with solving problems.

For MPP001 step 1 should be straightforward. As you get more ambitious with programming it can be tricky to define (and understand sufficiently well) what it is you want to do. Questions you should ask yourself include:

- What is the starting point (what information do I know)?
- What result(s) should be returned/output?
- What calculations etc are necessary
- Are there any limits on the input/output?

As you gain experience with programming you will develop you own list and learn to adjust it to the situation.

Step 2 is a **vital step** that has to happen before you begin to write the code. With a little practice you will be able to do this step in your head for simple problems. However, for anything involving more than a handful of lines of code it is best to write out the steps and refine them.

Of course, there are other stages – testing, optimisation/refinement, etc. but for now these are enough to get going.

8.1.1 Example: how many cans of paint?

You are tasked with working out how many cans of paint are required to coat the sides and top of a tank of diameter $5m$ and height $8m$ if each can covers $2.5m^2$.

Step one: Inputs: diameter, Height, coverage per can. (limits: all should be above zero)

Outputs: Number of cans (limits: should be an integer)

Equations: the surface area of a cylinder is given by:

$$\pi \times \text{diameter} \times \text{height}$$

and the area of an end is:

$$\frac{\pi \times \text{diameter}^2}{4}$$

the number of cans will be given by:

$$N = \frac{\text{total surface area}}{\text{coverage per can}}$$

Step two A first stage-wise step through of the process could be:

```
1. Store input values in sensibly named variables
2. Calculate number of cans
3. Output results
```

This could usefully be refined to:

```
1. Store input values in sensibly named variables
    * Diameter = 5, Height = 8, Coverage = 2.5
2. Calculate number of cans
    * Ncans = (Surface area)/Coverage
3. Output results
    * Ncans needs to be rounded up
```

This is about ready to turn into python code:

```
# code to calculate number of cans required to paint sides and top of
import math as math # provides math functions and constants
# define values. Names should be self explanatory
Diameter = 5.0
Height = 8.0
Coverage = 2.5
NumCans = 0. # make sure that start from zero (good practice)
# Now calculate the number of cans required
# divide surface area of cylinder side + top by coverage per can
NumCans = (math.pi*Diameter*Height + math.pi*Diameter**2/4.)/Coverage
# Output Result
print(f'You require {math.ceil(NumCans)} cans of paint') # ceil round
```

Notice the use of comments (anything after a # is a comment and does nothing code wise). These are a way to annotate the code so that it is easier to follow. It is difficult to have too many comments in your code especially if it contains more than a couple of active lines of code.

Now more complex problems will require more stages and you may go through three or four refinement stages before starting to convert to code. It is tempting to shortcut this, but you'll be a better programmer for taking the extra time. In fact you'll probably take less time in the end as there'll be less wrong with your code when you write it. Also, you may find that more refinement still is needed as you start the coding – don't worry about this it is bound to happen. As you get more practiced you will get used to making the judgement.

> **!** Important
>
> You may feel that you want to jump into coding after Stage One and for simple problems this may work. However, it is good to get into the habit of doing both steps you will benefit in the medium term–especially if you want to evidence code development for your project and this will be expected for the main assessments on this module.

9 Lab 2 The building blocks of computer code

In order to get a program with a useful level of complexity there are a few key building blocks. These are common to most computer languages (there will be syntax differences–for example python uses indentation of code to separate blocks whilst many others use curly brackets i.e. { }.

The building blocks key to introductory python programming are described below.

9.1 Conditional statements

These check whether a condition is met and perform different code based on this check. The classic structure is the `if` statement which will execute the following code if the condition tested evaluates to TRUE. `if` statements can be extended in a number of ways: an `else` block adds code that is only run if the condition is not met. In python there is also an `elif` command that can be thought of as 'else if...'

```
# if example
def Greater(x,y): # returns greater of x or y
    if x > y:
        return x
    elif y > x:
        return y
    else: return x # x&y must be equal, does not matter which is retu

print(Greater(4,5))
```

```
5
```

9.2 Loops

These are used to repeat a block of code either a fixed number of times (usually the `for` loop) or until some condition is met (`while` loop).

9.2.1 for loops

In most computer languages the for-loop increments/decrements a numeric variable through a range of values. Here is a C example

```
// Loop in C language - DO NOT copy to python
for (int i = 1; i < 11; ++i)
    // i goes from 1 to 10 in steps of 1
  {
    printf("%d ", i); // print current value of i
  }
```

Do not try and run this code in Jupyter (or any other python system) Hopefully you can see that it is designed to print out the integers from 1 to 10.

In python the loop goes through a list of values. The must all be the same type, but beyond that there are very few limits. So this example prints out the names of the primary colours:

```
colours = ['red','green','blue'] # a list of colours
for colour in colours: # start of the loop. The variable colour conta
    print(f'{colour}')
```

The C example above could be reproduced in python with something like:

```
for i in range(11): # range produces a list of integers from 0 up to
    print(i)
```

range() is a function that returns a list of numbers. It defaults to starting at 0, but this can be changed (e.g. `range(1,11)` would give 1, 2, ..., 10).

> **Tip**
>
> Loops are often used for calculating sums, products, etc. make sure the variable that you are storing the result in starts from where it is supposed to. For sums (`i = i+NextValue`) this will usually be zero. For products (`y = y*NextValue`) one is a better choice.

9.2.2 while loops

While loops are another way to repeat a block of code. The loop operates as long as a condition is true. The following example produces the same output as the `for` loop above.

```
i = 0 # start value
while i < 11: # loop with condition
    i += 1 # adds 1 to the value of i
    print(f'i = {i}')
```

> **Check the conditions**
>
> It is easy to get the stopping conditions of a while loop wrong and if you're lucky the loop will end sooner than you expect allowing you to see that there is a problem. **However** it often happens that the stop conditions are never met and the loop attempts to go on until the end of time. Forgetting to increment the loop variable is the most common mistake, but there are plenty of others - be vigilent!

9.2.3 `break` and `continue`

Python provides functions to alter a loop's operation. These work in both `for` and `while` structures.

`break` stops a loop. The following loop will stop when `i` reaches the value 5:

```
for i in range(1,8): # range produces a list of integers
    if i == 5: break
    print(i)
```

```
1
2
3
4
```

Quick question How would you change the loop above so that it printed out the value 5 before stopping?

`continue` goes to the next iteration of the loop rather than stopping the loop entirely:

```
i = 1 # start value
while i < 9: # loop with condition
    i += 1 # adds 1 to the value of i
    if i == 5: continue
    print(f'i = {i}')
```

```
i = 2
i = 3
i = 4
i = 6
i = 7
i = 8
i = 9
```

9.3 Functions

We've met these before, but for completeness here is a quick reminder

These are blocks of code that you send values to and the block returns one or more values. `print()` is an example of a built-in function. You can add your own functions to python code either by importing them from another file/script or by defining them in the script you are working on.

A minimal python example would be:

```
def HelloWorld():
    print('Hello world!')
```

Note the code after the `def` line is indented – this is how python tracks the extent of the current code block. You can see a similar use of indenting in the for loop above.

The brackets after `HelloWorld` can take the names of arguments (parameters) that are to be passed to the function. E.g.

```
def Quotient(x,y):
    print(x/y)
```

which could be called by `Quotient(2,3)`. Note the order of arguments matters.

You can give arguments default values:

```
def Sum(x,y=3):
    return(x+y)
```

Calling this with `Sum(4)` will return `7` whereas `Sum(4,2)` will return `6`. Note all arguments after the first with a default value must have default values.

9.4 Finding out more about python

An excellent resource for getting started in python is week one of this online course: introPy

There are lots of other learning resources available via the internet. LinkedIn Learning (for which you have access via the University) has lots of professionally created courses to choose from and Google and/or YouTube may have one or two things of interest as well.

10 Week 2 Lab

the exercises for the week 2 lab session are based on the material in the section “The building blocks of computer code”. They are designed to help you learn the fundemental structures that make up the vast majority of computer programs. You are **strongly** advised to begin to apply the method given in the “Programming (in python)” section. The method described will help you solve almost every computing problem (and many other engineering problems). If you use the method for the other assessments in the module then you are unlikelu to have diffculty demonstrating that you have met the GenAI useage requirements for the module.

10.1 Actions

- Review the sections “The building blocks of computer code” and “Programming (in python)” in the python notes.
- Attempt the problems in the Lab 2 notebook

11 Lab 4: Engineering Data Analysis with Pandas

Last time out we looked at reading numerical data contained in CSV files into numpy and performing some analysis on the datasets. Numpy excels at this sort of operation. Where it struggles is mixed datasets i.e. data with columns of different types, especially non-numeric. These mixed data are often referred to as structured data. Such data are remarkably common - for a lot of work we can use Excel, but for large data sets we need a way to handle this in python.

11.1 Pandas

Pandas (link: Documentation here) is an extremely widely used python library for handling/analysing structured data. For many of its operations it uses numpy but does all the preparitory work for you so that your code remains readable. Other libraries for structured data are available (Polars, Dask, PySpark, etc.) in some applications these have advantages, but Pandas is the market leader and many of the alternatives essentially copy the syntax of pandas and their advantages come from how they work in the background. We are going to work with Pandas here.

Numpy is built around the array structure. Pandas is built round a data structure known as a `dataframe`. Users of `R` may have cone across this concept with that language. A dataframe is a data set that can have columns of data of different types, the columns can have titles.

11.2 Getting started

We can import pandas in the usual way. It is usually given the shortname `pd`.

```
import pandas as pd
```

Let's import pandas and then import a file from last week's lab session:

```
import pandas as pd
from pathlib import Path
# Resolve absolute path to the dataset assuming a 'data' folder next
data_dir = Path("data").resolve() # the path to the data folder
file_path = data_dir / "L3DataTensile.csv" # specify which file in th

df = pd.read_csv(file_path)  # a dataframe called df - this is where
```

Notice that there was no skipping of lines - this is possible, but in this data set we can just use the data as it comes.

To get a quick look at the top of the dataframe use `dataframeName.head()`. This gives a nicely formatted view of the first few rows of the dataframe:

```
df.head()
```

	Force(N)	Distance(mm)
0	650.5	0.00000
1	650.5	0.00000
2	650.5	0.00000
3	650.5	0.00275
4	707.0	0.00825

Notice we now have an index column and the data columns have titles from the original data.

Pandas can read and write (create) files of many different types - including Excel files and tables. A quick introduction to Pandas and files is here (link: Pandas: getting started with files).

11.2.1 A bit of computing terminology: methods

Python is an Object Oriented Programming (OOP) language. For this module we will generally not worry about what this means. For this module the bit we need to know is that for non-OOP languages variables are places that information is stored whereas OOP variables - often referred to as objects - have the information plus a range of built-in functionality - known as methods. In the above `read_csv()` and `head()` are methods that Pandas dataframes possess. Where ever you see something of the form: `var_name.function_name()` then `function_name()` is a method of the object `var_name`.

11.2.1.1 Why is this important?

The methods in pandas do a lot of work for you. One key advantage that they bring is that most methods will work on the dataframe (or defined subsection of) without having to worry about loops etc to go through each row. In fact, if you do attempt to loop through a dataframe you are likely to make your code several orders of magnitude slower. This might not matter for a few tens of data, but it rapidly becomes an issues as the size of the data increases.

11.2.2 back to the code.

The method `tail()` displays the last few lines of the dataframe:

```
df.tail(10) # look at last 10 lines of the dataframe
```

	Force(N)	Distance(mm)
5123	25196.5	28.38375
5124	25071.0	28.38925
5125	25071.0	28.39550
5126	24896.5	28.40100
5127	24722.0	28.40675
5128	24630.0	28.41275
5129	24428.5	28.41850
5130	24428.5	28.42475
5131	24273.5	28.43100
5132	383.5	28.43650

Another method gives summary statistics (where possible)

```
df.describe()
```

	Force(N)	Distance(mm)
count	5133.000000	5133.000000
mean	27270.387882	13.705570
std	6359.543871	8.281797
min	383.500000	0.000000
25%	25019.500000	6.451750
50%	29615.500000	13.378250
75%	31512.500000	20.907500

	Force(N)	Distance(mm)
max	31953.000000	28.436500

11.3 Filtering data

Pandas uses a square bracket syntax to access different parts of a dataframe. At its simplest it is similar to the notation used for numpy, but it is much more powerful.

A single column in a dataframe can be obtained by referring to its name:

```
force = df["Force(N)"]
force.head()
```

```
0    650.5
1    650.5
2    650.5
3    650.5
4    707.0
Name: Force(N), dtype: float64
```

Let's get the rows for which the force generated is less than 1kN:

```
below1k = df[df["Force(N)"] < 1000] # filter rows where the force is
below1k.head()
```

	Force(N)	Distance(mm)
0	650.5	0.00000
1	650.5	0.00000
2	650.5	0.00000
3	650.5	0.00275
4	707.0	0.00825

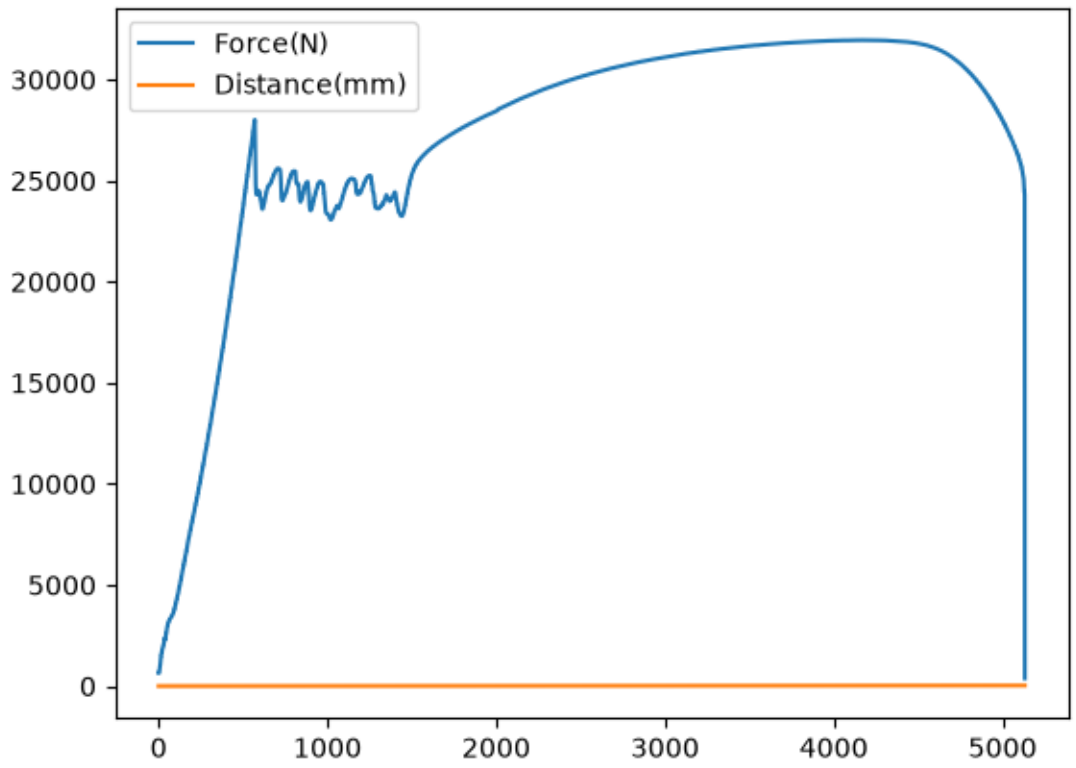
Other conditional tests can be used >, ==, etc.

There's more on filtering, reshaping, etc of dataframes here (link: Intro to slicing dataframes

11.4 Plotting with Pandas

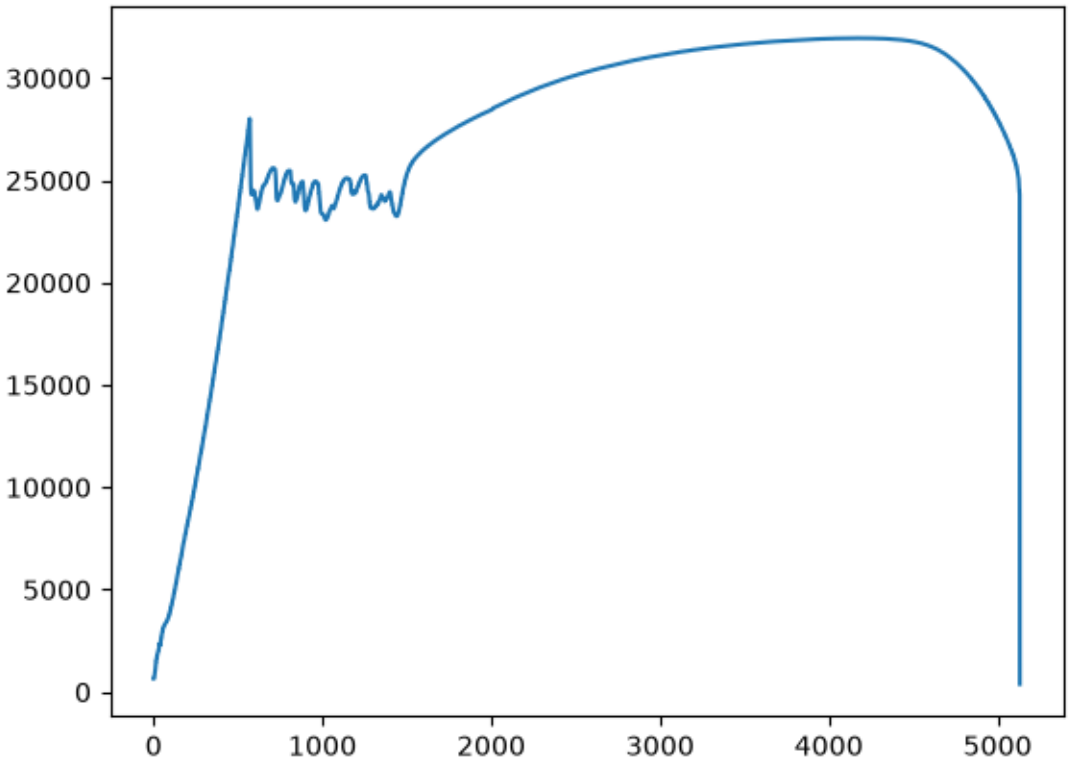
Since visualisation is a key partof data anaysis Pandas comes with some decent tools “out of the box”.

```
df.plot()
```



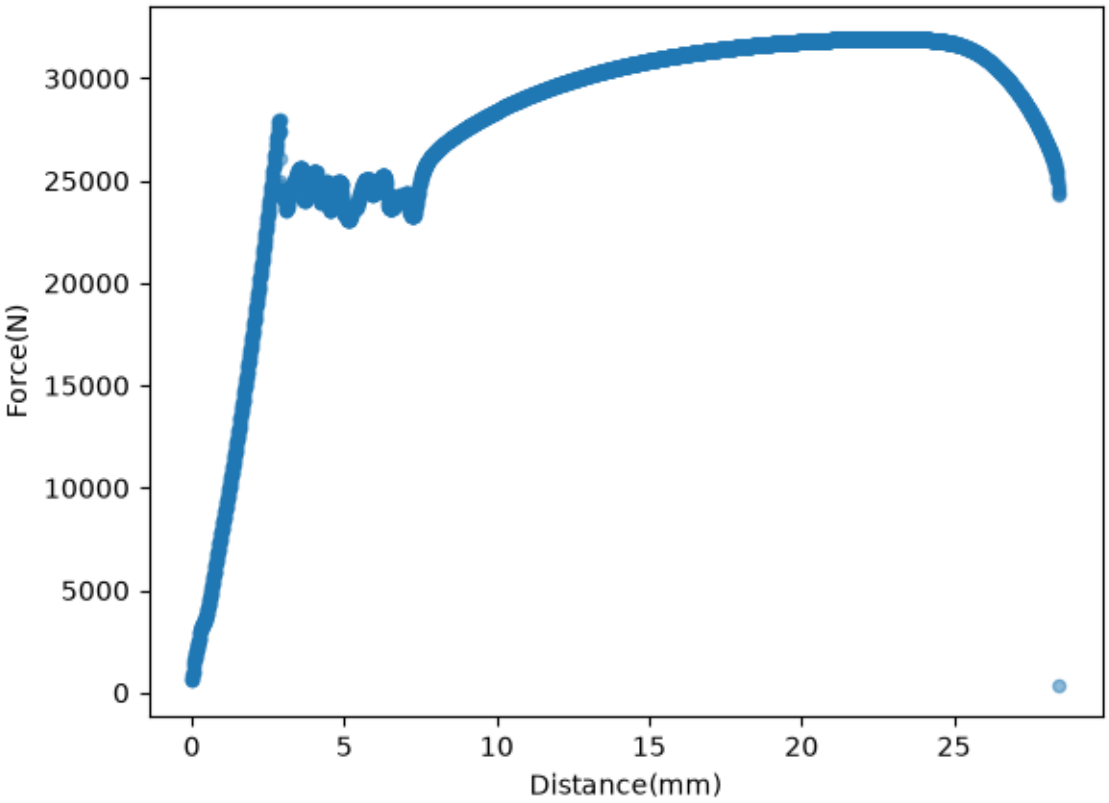
That’s not the most useful plot. Lets focus on the force data:

```
df["Force(N)"].plot() # use a method on a subset of a dataframe
```



That's better, but we can do more:

```
df.plot.scatter(x="Distance(mm)", y="Force(N)", alpha=0.5)
```



Notice in the last bit of python code there are two methods `plot` and `scatter`. The `plot` method gives a line plot by default, but it has other methods associated with it. In this case `scatter()` was used.

Pandas is capable of a a wide range of plot types. More details here: (link: Intro to Pandas plotting

As mentioned in last week's tutorial session the library seaborn brings lots of plot types (with pretty default settings). It works well with both numpy and pandas.

11.5 Accessing dataframe elements

We can access numpy array elements using numbers in square brackets e.g.

```
numpy_array_name[23][531]
```

It is slightly more complicated in pandas. In pandas there are two main ways to access specific (sets of) cells in a dataframe.

11.5.1 loc[]

The `loc[]` method provides a way to access cells using the labels of a dataframe. To see how this works we are going to use a dataframe borrowed from link: geeksforgeeks

```
# creating a sample dataframe
data = pd.DataFrame({'Brand': ['Maruti', 'Hyundai', 'Tata',
                               'Mahindra', 'Maruti', 'Hyundai',
                               'Renault', 'Tata', 'Maruti'],
                     'Year': [2012, 2014, 2011, 2015, 2012,
                              2016, 2014, 2018, 2019],
                     'Kms Driven': [50000, 30000, 60000,
                                    25000, 10000, 46000,
                                    31000, 15000, 12000],
                     'City': ['Gurgaon', 'Delhi', 'Mumbai',
                              'Delhi', 'Mumbai', 'Delhi',
                              'Mumbai', 'Chennai', 'Ghaziabad'],
                     'Mileage':  [28, 27, 25, 26, 28,
                                  29, 24, 21, 24]})

# displaying the DataFrame
display(data) # show all of the dataframe
```

	Brand	Year	Kms Driven	City	Mileage
0	Maruti	2012	50000	Gurgaon	28
1	Hyundai	2014	30000	Delhi	27
2	Tata	2011	60000	Mumbai	25
3	Mahindra	2015	25000	Delhi	26
4	Maruti	2012	10000	Mumbai	28
5	Hyundai	2016	46000	Delhi	29
6	Renault	2014	31000	Mumbai	24
7	Tata	2018	15000	Chennai	21
8	Maruti	2019	12000	Ghaziabad	24

```
display(data.loc[1: 3]) # use the index to give a range of rows
```

	Brand	Year	Kms Driven	City	Mileage
1	Hyundai	2014	30000	Delhi	27

	Brand	Year	Kms Driven	City	Mileage
2	Tata	2011	60000	Mumbai	25
3	Mahindra	2015	25000	Delhi	26

```
display(data.loc[(data.Brand == 'Maruti')]) # select a brand
```

	Brand	Year	Kms Driven	City	Mileage
0	Maruti	2012	50000	Gurgaon	28
4	Maruti	2012	10000	Mumbai	28
8	Maruti	2019	12000	Ghaziabad	24

```
display(data.loc[(data.Brand == 'Maruti') & (data.Mileage == 24)]) #
```

	Brand	Year	Kms Driven	City	Mileage
8	Maruti	2019	12000	Ghaziabad	24

11.5.2 iloc[]

The `iloc[]` method provides something close to numpy style element access i.e. integers are used to specify the a cell (or range of).

The simplest form takes a list of integers that specify first the row and then the column.

Remember that python indices start from 0.

e.g.

```
display(data.iloc[1,3])
```

```
'Delhi'
```

11.6 Working with numpy

Many of the methods that pandas provides use numpy behind the scenes. You often will need to create numpy arrays from a dataframe (or part of a dataframe) to use with another library that expects to get a numpy array e.g. scikit-learn. To make this easy pandas has the method `to_numpy()`

```
df.to_numpy() # returns a 2D numpy array
```

```
array([[  650.5    ,     0.     ],
       [  650.5    ,     0.     ],
       [  650.5    ,     0.     ],
       ...,
       [24428.5    ,    28.42475],
       [24273.5    ,    28.431  ],
       [  383.5    ,    28.4365 ]], shape=(5133, 2))
```

```
df["Force(N)"].to_numpy() # returns 1D numpy array
```

```
array([  650.5,   650.5,   650.5, ..., 24428.5, 24273.5,   383.5],
      shape=(5133,))
```

11.7 Creating copies

Dataframes have methods for creating copies of themselves. For non OOP languages a line such as

`x = y`

will put a copy of whatever is stored in `y` in a new variable `x` and these variables are independent of each other. With OOP's objects life is not so simple. The syntax above will work with Pandas objects, but there's more going on than we might expect. Initially `x` will just provide a way of accessing the same memory that is used to store `y` - it is what is known as a shallow copy. As soon as you attempt to change the value of any part of `x` python/pandas will make a copy of the data and associate that copy with `x` only. This new copy is a deep copy. A deep copy creates a new object, this is much less confusing when you are getting started, but given that ojects can be massive (many times the size of the memory in your computer) this can cause significant performance issues. In your programming you will almost always want a deep copy and this is what pandas defaults to. Behind the scenes for things like sending objects to functions etc. python/pandas is likely to use a shallow copy. The code below shows how to make deep and shallow copies.

```
# use df from above

dfNew1 = df.copy()              # deep copy
dfNew2 = df          # shallow copy n.b. dfNew2 = df.copy(False) also
```

References